

C++ ET LES BIBLIOTHÈQUES GRAPHIQUES

Cours Complet avec Projet Bancaire

Niveau : Licence / Master Informatique
Prérequis : Notions de base en C++
Professeur : Elias BADJI

MODULE 1 — Introduction à C++ et aux Interfaces Graphiques

1.1 Rappels C++ essentiels

C++ est un langage compilé, multi-paradigme (procédural, orienté objet, générique). Avant d'aborder les bibliothèques graphiques, il est indispensable de maîtriser les concepts suivants.

1.1.1 Gestion de la mémoire

La mémoire en C++ se divise en trois zones principales :

- Stack (pile) : variables locales, allocation automatique
- Heap (tas) : allocation dynamique avec new / delete
- Segment statique : variables globales et statiques

Exemple — Allocation dynamique

```
// Allocation sur le heap
int* tableau = new int[100];
// ... utilisation ...
delete[] tableau; // Libération obligatoire !

// Avec les smart pointers (C++11) — recommandé
#include <memory>
std::unique_ptr<int[]> tableau2 = std::make_unique<int[]>(100);
// Libération automatique quand le pointeur sort de portée
```

1.1.2 Programmation Orientée Objet

Les piliers de la POO en C++ sont indispensables pour structurer une application graphique.

Exemple — Classe de base pour un élément graphique

```
class Widget {
protected:
    int x, y, largeur, hauteur;
    std::string nom;
public:
    Widget(int x, int y, int l, int h, std::string n)
        : x(x), y(y), largeur(l), hauteur(h), nom(n) {}

    virtual void dessiner() = 0; // méthode pure
    virtual void gererEvenement() {} // méthode virtuelle
    virtual ~Widget() {} // destructeur virtuel

    // Accesseurs
    int getX() const { return x; }
    int getY() const { return y; }
};
```

1.1.3 Templates et STL

La Standard Template Library fournit des conteneurs essentiels pour les applications graphiques :

Conteneur	Usage typique en GUI	Complexité
std::vector<T>	Liste de widgets, éléments d'un menu	O(1) accès

<code>std::map<K,V></code>	Table des événements (touche → action)	$O(\log n)$
<code>std::queue<T></code>	File d'événements à traiter	$O(1)$ push/pop
<code>std::string</code>	Textes, libellés, identifiants	$O(n)$ concat
<code>std::shared_ptr<T></code>	Partage de ressources (images, polices)	$O(1)$

1.2 Panorama des bibliothèques graphiques C++

Plusieurs bibliothèques permettent de créer des interfaces graphiques en C++. Voici une comparaison des principales.

Bibliothèque	Type	Licence	Plateformes	Usage
Qt	Framework complet	LGPL / Commercial	Win / Mac / Linux / Mobile	Applications professionnelles
SFML	Multimedia 2D	zlib/libpng	Win / Mac / Linux	Jeux, animations
SDL2	Bas niveau multimedia	zlib	Toutes + Web	Jeux, émulateurs
OpenGL + GLFW	3D bas niveau	Libre	Toutes	3D, visualisation
wxWidgets	GUI natif	wxWindows	Win / Mac / Linux	Bureautique
FLTK	GUI léger	LGPL	Win / Mac / Linux	Outils embarqués
Dear ImGui	GUI immédiat	MIT	Toutes	Outils, débogage

Conseil : Pour ce cours, nous utilisons principalement Qt (interface professionnelle) et SFML (animation et graphisme 2D). Ces deux bibliothèques couvrent 90% des besoins applicatifs.

MODULE 2 — SFML : Simple and Fast Multimedia Library

2.1 Installation et configuration

SFML s'installe facilement sur toutes les plateformes majeures.

Installation Linux/Ubuntu

```
# Installation via apt
sudo apt-get install libsFML-dev

# Compilation d'un programme SFML
g++ main.cpp -o programme -lsfml-graphics -lsfml-window -lsfml-system
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(MonApp)
find_package(SFML 2.5 COMPONENTS graphics window system REQUIRED)
add_executable(MonApp main.cpp)
target_link_libraries(MonApp sfml-graphics sfml-window sfml-system)
```

2.2 Structure d'une application SFML

Toute application SFML suit la même boucle principale : initialisation, boucle d'événements, mise à jour, rendu.

Structure de base — Fenêtre SFML

```
#include <SFML/Graphics.hpp>

int main() {
    // 1. Création de la fenêtre
    sf::RenderWindow fenetre(sf::VideoMode(800, 600),
                             "Mon Application");
    fenetre.setFramerateLimit(60);

    // 2. Boucle principale
    while (fenetre.isOpen()) {
        // 2a. Gestion des événements
        sf::Event evenement;
        while (fenetre.pollEvent(evenement)) {
            if (evenement.type == sf::Event::Closed)
                fenetre.close();
            if (evenement.type == sf::Event::KeyPressed &&
                evenement.key.code == sf::Keyboard::Escape)
                fenetre.close();
        }

        // 2b. Logique métier (mise à jour)
        // ...

        // 2c. Rendu
        fenetre.clear(sf::Color(30, 30, 50)); // Fond sombre
        fenetre.draw(...);
        fenetre.display();
    }
    return 0;
}
```

2.3 Formes géométriques et dessin

SFML propose un ensemble riche de primitives graphiques prêtes à l'emploi.

Formes géométriques SFML

```
// Rectangle (bouton, cadre)
sf::RectangleShape rect(sf::Vector2f(200, 50));
rect.setFillColor(sf::Color(70, 130, 180));
rect.setOutlineThickness(2);
rect.setOutlineColor(sf::Color::White);
rect.setPosition(100, 100);

// Cercle (avatar, indicateur)
sf::CircleShape cercle(40);
cercle.setFillColor(sf::Color(220, 80, 80));
cercle.setPosition(350, 100);

// Ligne (graphique)
sf::Vertex ligne[] = {
    sf::Vertex(sf::Vector2f(50, 300), sf::Color::Yellow),
    sf::Vertex(sf::Vector2f(750, 300), sf::Color::Yellow)
};

// Texte
sf::Font police;
police.loadFromFile("arial.ttf");
sf::Text texte("Bonjour !", police, 30);
texte.setFillColor(sf::Color::White);
texte.setPosition(300, 250);

// Dans la boucle de rendu :
fenetre.draw(rect);
fenetre.draw(cercle);
fenetre.draw(ligne, 2, sf::Lines);
fenetre.draw(texte);
```

2.4 Gestion des événements

La gestion des événements est au cœur de toute interface interactive. SFML propose un système simple et complet.

Types d'événements SFML

```
while (fenetre.pollEvent(ev)) {
    switch (ev.type) {
        case sf::Event::Closed:
            fenetre.close(); break;

        case sf::Event::KeyPressed:
            if (ev.key.code == sf::Keyboard::Left)  deplacerGauche();
            if (ev.key.code == sf::Keyboard::Right) deplacerDroite();
            break;

        case sf::Event::MouseButtonPressed:
            if (ev.mouseButton.button == sf::Mouse::Left) {
                int mx = ev.mouseButton.x;
                int my = ev.mouseButton.y;
                gererClic(mx, my);
            }
    }
}
```

```
        break;

    case sf::Event::TextEntered:
        if (ev.text.unicode < 128) // ASCII
            champTexte += (char)ev.text.unicode;
        break;

    case sf::Event::Resized:
        sf::FloatRect vue(0, 0, ev.size.width, ev.size.height);
        fenetre.setView(sf::View(vue));
        break;
}
}
```

2.5 Chargement d'images et textures

Images et sprites

```
#include <SFML/Graphics.hpp>

// Charger une texture depuis un fichier
sf::Texture texture;
if (!texture.loadFromFile("logo_banque.png")) {
    // Erreur de chargement
    return -1;
}

// Créer un sprite à partir de la texture
sf::Sprite sprite;
sprite.setTexture(texture);
sprite.setPosition(50, 50);
sprite.setScale(0.5f, 0.5f); // Réduire à 50%

// Transparence
sprite.setColor(sf::Color(255, 255, 255, 200)); // alpha = 200/255

fenetre.draw(sprite);
```

MODULE 3 — Qt : Framework GUI Professionnel

3.1 Architecture Qt

Qt est bien plus qu'une bibliothèque graphique : c'est un framework complet qui intègre la gestion de la mémoire, le réseau, les bases de données, les tests, l'internationalisation, et bien plus.

Module Qt	Description	Classes principales
Qt Core	Fonctionnalités non graphiques	QObject, QString, QList, QTimer
Qt Widgets	Widgets classiques bureau	QMainWindow, QPushButton, QLabel
Qt GUI	Rendu 2D, polices, images	QPainter, QFont, QImage, QPixmap
Qt Network	HTTP, TCP, UDP	QNetworkAccessManager, QTcpSocket
Qt SQL	Accès bases de données	QSqlDatabase, QSqlQuery
Qt Charts	Graphiques et courbes	QChart, QLineSeries, QPieSeries
Qt Quick/QML	Interfaces modernes	QML, JavaScript embarqué

3.2 Le système de signaux et slots

C'est la fonctionnalité la plus importante de Qt. Les signaux et slots permettent une communication découplée entre objets, remplaçant les callbacks traditionnels.

Principe : Un signal est émis quand quelque chose se produit (clic, texte modifié). Un slot est une fonction appelée en réponse. La connexion est établie par `connect()`.

Exemple — Signaux et Slots

```
#include <QObject>
#include <QPushButton>
#include <QLineEdit>

class CompteBancaire : public QObject {
    Q_OBJECT
public:
    explicit CompteBancaire(double solde, QObject* parent = nullptr)
        : QObject(parent), solde(solde) {}

signals:
    void soldeModifie(double nouveauSolde); // Signal émis
    void alerteSoldeInsuffisant();

public slots:
    void deposer(double montant) {
        if (montant > 0) {
            solde += montant;
            emit soldeModifie(solde); // Émet le signal
        }
    }
    void retirer(double montant) {
        if (montant > solde) { emit alerteSoldeInsuffisant(); return; }
        solde -= montant;
        emit soldeModifie(solde);
    }
}
```

```

    }

private:
    double solde;
};

// Connexion dans main :
CompteBancaire* compte = new CompteBancaire(1000.0);
QLabel* lblSolde = new QLabel();

// Quand le solde change, mettre à jour le label
QObject::connect(compte, &CompteBancaire::soldeModifie,
    [lblSolde](double s) {
        lblSolde->setText(QString("Solde : %1 FCFA").arg(s));
    });

```

3.3 Widgets essentiels Qt

Qt fournit une large gamme de widgets prêts à l'emploi. Voici les plus utilisés dans une application bancaire :

Widgets de base

```

#include <QApplication>
#include <QMainWindow>
#include <QWidget>
#include <QPushButton>
#include <QLabel>
#include <QLineEdit>
#include <QComboBox>
#include <QTableWidget>
#include <QMessageBox>

// Bouton
QPushButton* btn = new QPushButton("Valider");
btn->setStyleSheet("background-color: #1B3A6B; color: white; "
    "border-radius: 5px; padding: 8px;");

// Champ de saisie
QLineEdit* montantEdit = new QLineEdit();
montantEdit->setPlaceholderText("Entrez le montant...");
montantEdit->setValidator(new QDoubleValidator(0, 1e9, 2)); // Validation

// Liste déroulante
QComboBox* typeOp = new QComboBox();
typeOp->addItem("Virement", "Dépôt", "Retrait");

// Tableau de données
QTableWidget* table = new QTableWidget(10, 4);
table->setHorizontalHeaderLabels({"Date", "Type", "Montant", "Solde"});

```

3.4 Layouts et mise en page

Les layouts gèrent automatiquement le positionnement et le redimensionnement des widgets.

Layouts Qt

```

#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QGridLayout>

```



```
#include <QFormLayout>

// Layout vertical (empilement)
QVBoxLayout* vLayout = new QVBoxLayout();
vLayout->addWidget(new QLabel("Compte :"));
vLayout->addWidget(new QLineEdit());
vLayout->addSpacing(20);
vLayout->addWidget(new QPushButton("Valider"));

// Layout horizontal
QHBoxLayout* hLayout = new QHBoxLayout();
hLayout->addWidget(new QPushButton("Annuler"));
hLayout->addStretch(); // Espace extensible
hLayout->addWidget(new QPushButton("Confirmer"));

// Formulaire (label + champ, alignés)
QFormLayout* form = new QFormLayout();
form->addRow("Nom :", new QLineEdit());
form->addRow("Solde initial :", new QLineEdit());
form->addRow("Type de compte :", new QComboBox());

// Grille
QGridLayout* grid = new QGridLayout();
grid->addWidget(btn1, 0, 0); // ligne 0, colonne 0
grid->addWidget(btn2, 0, 1); // ligne 0, colonne 1
grid->addWidget(btn3, 1, 0, 1, 2); // colspan 2
```

3.5 QPainter — Dessin personnalisé

QPainter permet de dessiner des graphiques personnalisés (courbes de solde, graphiques camembert, etc.) directement dans un widget.

Graphique personnalisé avec QPainter

```
#include <QWidget>
#include <QPainter>
#include <QVector>

class GraphiqueSolde : public QWidget {
    Q_OBJECT
public:
    explicit GraphiqueSolde(QWidget* parent = nullptr) : QWidget(parent) {}

    void setDonnees(QVector<double> valeurs) {
        this->valeurs = valeurs;
        update(); // Déclenche paintEvent
    }

protected:
    void paintEvent(QPaintEvent*) override {
        QPainter p(this);
        p.setRenderHint(QPainter::Antialiasing);

        int w = width(), h = height();
        // Fond dégradé
        QLinearGradient bg(0,0,0,h);
        bg.setColorAt(0, QColor(27, 58, 107));
        bg.setColorAt(1, QColor(10, 20, 40));
        p.fillRect(rect(), bg);

        if (valeurs.isEmpty()) return;
        double maxV = *std::max_element(valeurs.begin(), valeurs.end());
        double minV = *std::min_element(valeurs.begin(), valeurs.end());
```

```
// Tracer la courbe
QPen stylo(QColor(100, 180, 255), 2);
p.setPen(stylo);
int n = valeurs.size();
for (int i = 1; i < n; ++i) {
    int x1 = (i-1) * w / (n-1);
    int y1 = h - (int)((valeurs[i-1]-minV)/(maxV-minV) * (h-40)) - 20;
    int x2 = i * w / (n-1);
    int y2 = h - (int)((valeurs[i]-minV)/(maxV-minV) * (h-40)) - 20;
    p.drawLine(x1, y1, x2, y2);
}

private:
    QVector<double> valeurs;
};
```

MODULE 4 — Architecture d'une Application Graphique C++

4.1 Le patron MVC (Modèle-Vue-Contrôleur)

L'architecture MVC est essentielle pour séparer la logique métier de l'interface graphique.

Composant	Rôle	Exemple bancaire
Modèle (Model)	Données et logique métier pure	CompteBancaire, Transaction, Client
Vue (View)	Affichage, widgets, rendu	FenetreCompte, TableauTransactions
Contrôleur (Controller)	Reçoit événements, met à jour M et V	BancaireController

Structure MVC — Exemple bancaire

```
// — MODÈLE —————
class Client {
    QString nom, prenom, numeroClient;
    QVector<CompteBancaire*> comptes;
public:
    void ajouterCompte(CompteBancaire* c) { comptes.append(c); }
    double soldeTotal() const;
};

// — VUE —————
class VueTableauBord : public QMainWindow {
    Q_OBJECT
public:
    VueTableauBord(QWidget* parent = nullptr);
    void afficherSolde(double s);
    void afficherTransactions(QVector<Transaction> liste);
signals:
    void virementDemande(QString dest, double montant);
    void retraitDemande(double montant);
};

// — CONTRÔLEUR —————
class BancaireController : public QObject {
    Q_OBJECT
    Client* modele;
    VueTableauBord* vue;
public:
    BancaireController(Client* m, VueTableauBord* v, QObject* parent = nullptr);
public slots:
    void gererVirement(QString dest, double montant);
    void gererRetrait(double montant);
};
```

4.2 Gestion des ressources et performances

Une application graphique doit être fluide et réactive. Voici les bonnes pratiques à respecter.

- Ne jamais bloquer le thread principal (UI thread) avec des opérations longues
- Utiliser QThread ou std::thread pour les calculs en arrière-plan
- Mettre en cache les textures/images déjà chargées

- Limiter le taux de rafraîchissement (60 FPS est suffisant pour une GUI bancaire)
- Utiliser les signaux asynchrones pour les appels réseau et base de données

Thread séparé pour opération longue

```
#include <QThread>
#include <QFuture>
#include <QtConcurrent>

// Méthode recommandée avec QtConcurrent
void chargerHistoriqueTransactions(QString compteId) {
    // Exécuter dans un thread séparé
    QFuture<QVector<Transaction>> futur = QtConcurrent::run([=]() {
        // Opération longue (accès BDD, réseau...)
        return baseDesDonnees.getTransactions(compteId);
    });

    // Quand c'est fini, mettre à jour l'UI dans le thread principal
    QFutureWatcher<QVector<Transaction>>* watcher = new QFutureWatcher<>(this);
    connect(watcher, &QFutureWatcher<QVector<Transaction>>::finished,
            this, [this, watcher]() {
                vue->afficherTransactions(watcher->result());
                watcher->deleteLater();
            });
    watcher->setFuture(futur);
}
```

MODULE 5 — Introduction à OpenGL avec GLFW

5.1 Concepts fondamentaux OpenGL

OpenGL est une API de rendu 3D bas niveau. Elle est utile pour les visualisations avancées (graphiques 3D, animations complexes, tableaux de bord immersifs).

Concept	Description
Vertex Buffer Object (VBO)	Stocke les données de vertices (positions, couleurs) sur le GPU
Vertex Array Object (VAO)	Mémoire la configuration d'un VBO
Shader	Programme exécuté sur le GPU (GLSL)
Fragment Shader	Calcule la couleur de chaque pixel
Vertex Shader	Transforme les positions des vertices
Framebuffer	Tampon de rendu (ce qui s'affiche à l'écran)

Programme OpenGL minimal avec GLFW

```
#include <GL/glew.h>
#include <GLFW/glfw3.h>

int main() {
    // Initialisation GLFW
    glfwInit();
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

    GLFWwindow* fenetre = glfwCreateWindow(800, 600,
                                           "Visualisation Bancaire 3D", NULL, NULL);
    glfwMakeContextCurrent(fenetre);

    glewInit(); // Charger les extensions OpenGL

    // Couleur de fond (bleu foncé, style bancaire)
    glClearColor(0.07f, 0.14f, 0.27f, 1.0f);

    while (!glfwWindowShouldClose(fenetre)) {
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
        // Dessin ici...
        glfwSwapBuffers(fenetre);
        glfwPollEvents();
    }
    glfwTerminate();
    return 0;
}
```

MODULE 6 — PROJET : Système de Gestion Bancaire Graphique

Ce projet constitue l'évaluation finale du cours. Il doit être réalisé en binôme en 2 à 3 semaines. Il couvre tous les modules du cours et intègre une interface graphique complète avec Qt ou SFML.

6.1 Présentation du projet

Titre : BankVision — Interface Graphique de Gestion Bancaire

BankVision est une application de bureau complète permettant de gérer des comptes bancaires, effectuer des opérations financières, et visualiser des statistiques en temps réel.

6.2 Cahier des charges fonctionnel

6.2.1 Module Authentification

- Écran de connexion avec champs login / mot de passe
- Hachage du mot de passe (SHA-256 via QCryptographicHash)
- Gestion des sessions (3 essais max, verrouillage temporaire)
- Interface visuelle : fond bleu nuit, logo banque, animation de chargement

6.2.2 Module Gestion des Comptes

- Création de compte (courant, épargne, professionnel)
- Affichage du solde en temps réel avec indicateur coloré (rouge < 0 , vert $> \text{objectif}$)
- Historique des 30 dernières transactions dans un tableau trié
- Export CSV des relevés de compte

6.2.3 Module Opérations Bancaires

- Dépôt d'espèces avec confirmation graphique animée
- Retrait avec vérification du solde disponible
- Virement interne et externe avec validation en 2 étapes
- Demande de prêt avec simulation de mensualités

6.2.4 Module Tableaux de Bord & Statistiques

- Courbe d'évolution du solde sur 12 mois (QPainter ou Qt Charts)
- Répartition des dépenses par catégorie (camembert)
- Comparaison recettes / dépenses (histogramme)
- Indicateurs clés : solde moyen, dépense max, taux d'épargne

6.2.5 Module Administration (Bonus)

- Liste de tous les clients et comptes
- Validation des demandes de prêt en attente
- Génération de rapport PDF mensuel

6.3 Architecture technique recommandée

Structure des fichiers du projet

```

BankVision/
├── CMakeLists.txt
├── main.cpp
├── models/
│   ├── Client.h / Client.cpp
│   ├── CompteBancaire.h / CompteBancaire.cpp
│   ├── Transaction.h / Transaction.cpp
│   ├── Pret.h / Pret.cpp
│   └── Banque.h / Banque.cpp
├── views/
│   ├── FenetreConnexion.h / .cpp
│   ├── FenetreTableauBord.h / .cpp
│   ├── FenetreOperations.h / .cpp
│   ├── FenetreStatistiques.h / .cpp
│   └── widgets/
│       ├── GraphiqueSolde.h / .cpp
│       ├── CamembertDepenses.h / .cpp
│       └── IndicateurSolde.h / .cpp
├── controllers/
│   ├── AuthController.h / .cpp
│   ├── CompteController.h / .cpp
│   └── StatController.h / .cpp
├── data/
│   ├── banque.db          (SQLite)
│   └── DataManager.h / .cpp
├── resources/
│   ├── images/
│   ├── styles/
│   │   └── style.qss      (feuille de style Qt)
│   └── resources.qrc

```

6.4 Implémentation — Code de démarrage

main.cpp — Point d'entrée

```

#include <QApplication>
#include <QFile>
#include "views/FenetreConnexion.h"
#include "data/DataManager.h"

int main(int argc, char* argv[]) {
    QApplication app(argc, argv);

    // Charger la feuille de style
    QFile styleFile(":/styles/style.qss");
    if (styleFile.open(QIODevice::ReadOnly)) {
        app.setStyleSheet(styleFile.readAll());
        styleFile.close();
    }

    // Initialiser la base de données
    DataManager::instance().initialiser("data/banque.db");

    // Afficher la fenêtre de connexion
    FenetreConnexion connexion;
    connexion.show();

    return app.exec();
}

```

CompteBancaire.h — Modèle principal

```

#pragma once
#include <QString>
#include <QVector>
#include <QDateTime>

enum class TypeCompte { COURANT, EPARGNE, PROFESSIONNEL };
enum class StatutCompte { ACTIF, BLOQUE, FERME };

struct Transaction {
    int id;
    QDateTime date;
    QString type;          // "depot", "retrait", "virement"
    double montant;
    double soldeApres;
    QString description;
};

class CompteBancaire {
public:
    CompteBancaire(const QString& iban, TypeCompte type,
                   double soldeInitial = 0.0);

    // Opérations
    bool deposer(double montant, const QString& desc = "");
    bool retirer(double montant, const QString& desc = "");
    bool virer(CompteBancaire& dest, double montant);

    // Accesseurs
    double getSolde() const { return solde; }
    QString getIBAN() const { return iban; }
    TypeCompte getType() const { return type; }
    StatutCompte getStatut() const { return statut; }
    QVector<Transaction> getHistorique(int n = 30) const;

    // Statistiques
    double getSoldeMoyen(int jours = 30) const;
    QVector<double> getSoldesMensuels(int mois = 12) const;

private:
    QString iban;
    double solde;
    TypeCompte type;
    StatutCompte statut;
    QVector<Transaction> historique;

    void enregistrerTransaction(const QString& t, double m, const QString& d);
};

```

style.qss — Thème visuel bancaire

```

/* Fenêtre principale */
QMainWindow {
    background-color: #0F1E3D;
}

/* Boutons primaires */
QPushButton[type="primary"] {
    background-color: #2E6DB4;
    color: white;
    border: none;
    border-radius: 6px;
    padding: 10px 20px;
    font-size: 14px;
}

```



```

        font-weight: bold;
    }
    QPushButton[type="primary"]:hover {
        background-color: #1B3A6B;
    }
    QPushButton[type="danger"] {
        background-color: #C0392B;
        color: white;
        border-radius: 6px;
        padding: 10px 20px;
    }

    /* Champs de saisie */
    QLineEdit {
        background-color: #1A2F5A;
        color: #E8F0FF;
        border: 1px solid #2E6DB4;
        border-radius: 4px;
        padding: 8px;
        font-size: 13px;
    }
    QLineEdit:focus { border-color: #5BA3E0; }

    /* Tableau des transactions */
    QTableWidget {
        background-color: #12244A;
        color: #D0E4FF;
        gridline-color: #1A3A7A;
        selection-background-color: #2E6DB4;
    }
    QHeaderView::section {
        background-color: #1B3A6B;
        color: white;
        padding: 6px;
        border: none;
        font-weight: bold;
    }

```

6.5 Fonctionnalité avancée — Graphique QPainter

GraphiqueSolde.cpp — Courbe animée

```

#include "GraphiqueSolde.h"
#include <QPainter>
#include <QPainterPath>
#include <QLinearGradient>
#include <algorithm>

void GraphiqueSolde::paintEvent(QPaintEvent*) {
    QPainter p(this);
    p.setRenderHint(QPainter::Antialiasing, true);

    int W = width(), H = height();
    const int MARGE = 40;

    // Fond
    QLinearGradient bg(0, 0, 0, H);
    bg.setColorAt(0.0, QColor(18, 36, 74));
    bg.setColorAt(1.0, QColor(10, 20, 40));
    p.fillRect(rect(), bg);

    if (valeurs.size() < 2) return;

    double maxV = *std::max_element(valeurs.begin(), valeurs.end());
    double minV = *std::min_element(valeurs.begin(), valeurs.end());
    if (maxV == minV) maxV = minV + 1;

```

```

    auto xPt = [&](int i) {
        return MARGE + (double)(i) * (W - 2*MARGE) / (valeurs.size()-1);
    };
    auto yPt = [&](double v) {
        return MARGE + (1.0 - (v - minV)/(maxV - minV)) * (H - 2*MARGE);
    };

    // Zone remplie sous la courbe
    QPainterPath aire;
    aire.moveTo(xPt(0), H - MARGE);
    for (int i = 0; i < valeurs.size(); ++i)
        aire.lineTo(xPt(i), yPt(valeurs[i]));
    aire.lineTo(xPt(valeurs.size()-1), H - MARGE);
    aire.closeSubpath();

    QLinearGradient fill(0, MARGE, 0, H - MARGE);
    fill.setColorAt(0.0, QColor(46, 109, 180, 120));
    fill.setColorAt(1.0, QColor(46, 109, 180, 0));
    p.fillPath(aire, fill);

    // Courbe
    QPainterPath courbe;
    courbe.moveTo(xPt(0), yPt(valeurs[0]));
    for (int i = 1; i < valeurs.size(); ++i)
        courbe.lineTo(xPt(i), yPt(valeurs[i]));
    p.setPen(QPen(QColor(100, 180, 255), 2.5));
    p.drawPath(courbe);

    // Points
    p.setBrush(QColor(100, 180, 255));
    for (int i = 0; i < valeurs.size(); ++i)
        p.drawEllipse(QPointF(xPt(i), yPt(valeurs[i])), 4, 4);
}

```

6.6 Grille d'évaluation

Critère	Barème	Détail
Interface graphique	5 points	Qualité visuelle, cohérence, thème
Architecture MVC	4 points	Séparation M/V/C, respect des patterns
Fonctionnalités	5 points	Toutes les opérations bancaires
Visualisations	3 points	Graphiques, courbes, statistiques
Code et qualité	2 points	Lisibilité, commentaires, pas de fuite mémoire
Rapport + Soutenance	1 point	Présentation orale, démonstration
BONUS — Administration	+2 points	Module admin complet
TOTAL	20 points	

6.7 Planning suggéré

Semaine	Objectifs
Semaine 1	Architecture projet, modèles de données (Client, Compte, Transaction), maquette UI
Semaine 2	Fenêtres de connexion et tableau de bord, opérations bancaires (dépôt, retrait, virement)
Semaine 3	Graphiques et statistiques, tests, polissage visuel, rapport et préparation soutenance

ANNEXES — Références et Ressources

A. Ressources en ligne

- Documentation Qt officielle : <https://doc.qt.io>
- SFML — Tutoriels officiels : <https://www.sfml-dev.org/tutorials/>
- cppreference.com — Référence C++ complète
- learnopengl.com — Tutoriels OpenGL progressifs
- GitHub — Chercher des projets C++ GUI open-source

B. Bibliothèques additionnelles utiles

Bibliothèque	Utilité	Installation
nlohmann/json	Parsing JSON (API bancaires)	vcpkg ou header-only
SQLiteCpp	Base de données embarquée	vcpkg / CMake FetchContent
OpenSSL	Chiffrement, HTTPS	sudo apt install libssl-dev
bcrypt / Crypto++	Hachage mots de passe	sudo apt install libcryptopp-dev
Boost.Asio	Réseau asynchrone	sudo apt install libboost-dev

C. Commandes de compilation fréquentes

Compilation
<pre># Programme SFML g++ -std=c++17 main.cpp -o app \ -lsfml-graphics -lsfml-window -lsfml-system # Projet Qt (qmake) qmake BankVision.pro && make # Projet Qt (CMake) mkdir build && cd build cmake .. -DCMAKE_BUILD_TYPE=Release make -j\$(nproc) # Vérifier les fuites mémoire (valgrind) valgrind --leak-check=full ./BankVision</pre>

Bon courage à tous les étudiants ! Ce cours est conçu pour vous donner les outils nécessaires pour développer des applications graphiques professionnelles en C++. Le projet bancaire vous permettra d'appliquer l'ensemble des concepts vus en cours dans un contexte réaliste et valorisant pour votre portfolio.